

Interactive Theorem Proving: Introduction

Formal Verification: 6CCS3VER

This Video

- What is interactive theorem proving?
- What are we doing over the next three lectures?
- What do I expect you to already know?
- Some Organisational notes

What is ITP'ing?

Formal proofs:

- Recall with me the following Aristotlian syllogism
 - 1: All men are mortal (Assume)
 - 2: Aristotle is a man (Assume)
 - 3: Aristotle is mortal (from 1, 2, and modus-ponens)
- Modus-ponens: if a implies b and a holds, then b holds
- I hope we all agree that this is an unambiguously correct proof

What is ITP'ing?

- Our objective: construct formal proofs
- Unambiguous, correct proofs:
 - Definitions' and theorem statements' meaning and well-formedness
 - Proof correctness
- All of that wrt precisely and concisely stated axioms

What is ITP'ing?

- Proofs and definitions are written in a formal language
- Meaning is defined by set of axioms, e.g.
 - Linear temporal logic axioms (recall from week 3 lecture)
 - More general: Zermelo-Fraenkel set theory:
 - * Extensionality: $\forall s, t. (\forall x. x \in s \leftrightarrow x \in t) \rightarrow s = t$
 - * Regularity: $\forall s. s \neq \emptyset \rightarrow \exists t. t \in s \wedge t \cap s = \emptyset$
 - * Schema:
 $\forall s, x_1, \dots, x_n. \exists t. \forall y. y \in t \leftrightarrow (y \in s \wedge \phi(s, x_1, \dots, x_n, y))$
 - * ...
 - Others formalisms
 - * Type theory, Peano Arithmetic, First-order logic, and Higher-order logic (HOL)

What is ITP'ing?

- Why formal proofs?
- Initially to “formalise” mathematics
 - E.g. Euclid's elements (300 B.C.) and principia mathematica [Russel and Whitehead 1910]

What is ITP'ing?

- Challenge: is the size of a formal proof
 - Russel and Whitehead prove that $1+1=2$ after 173 pages

*54.43. $\vdash :: \alpha, \beta \in 1 . \supset : \alpha \cap \beta = \Lambda . \equiv . \alpha \cup \beta \in 2$

Dem.

$$\begin{aligned} &\vdash . *54.26 . \supset \vdash :: \alpha = \iota'x . \beta = \iota'y . \supset : \alpha \cup \beta \in 2 . \equiv . x \neq y . \\ &[*51.231] \qquad \qquad \qquad \equiv . \iota'x \cap \iota'y = \Lambda . \\ &[*13.12] \qquad \qquad \qquad \equiv . \alpha \cap \beta = \Lambda \qquad (1) \end{aligned}$$

$$\begin{aligned} &\vdash . (1) . *11.11.35 . \supset \\ &\quad \vdash :: (\exists x, y) . \alpha = \iota'x . \beta = \iota'y . \supset : \alpha \cup \beta \in 2 . \equiv . \alpha \cap \beta = \Lambda \qquad (2) \end{aligned}$$

$$\vdash . (2) . *11.54 . *52.1 . \supset \vdash . \text{Prop}$$

From this proposition it will follow, when arithmetical addition has been defined, that $1 + 1 = 2$.

- Unrealistic for non-trivial proofs, due to size and tedium
- Never took-off, except as a theoretical idea

What is ITP'ing?

- Machines can check proofs, and generate them
 - Automated theorem provers
 - * model-checkers
 - * SAT-solvers
 - * first-order logic automatic theorem provers
 - Problem: does not scale
 - Interactive theorem provers (ITPs)
 - * e.g. Isabelle/HOL [Nipkow et al. 2002], HOL4 [Slind and Norrish 2008], Coq [Bertot and Casteran], Lean [de Moura]

What is ITP'ing?

```
| 1| theory Sqrt2
| 2| imports Main
| 3| begin
| 4| theorem sqrt2_not_rational:
| 5|   "sqrt (real 2)  $\notin$   $\mathbb{Q}$ "
| 6| proof
| 7|   let ?x = "sqrt (real 2)"
| 8|   assume "?x  $\in$   $\mathbb{Q}$ "
| 9|   then obtain m n :: nat where
|10|     sqrt_rat: "?x = real m / real n" and lowest_terms: "coprime m n"
|11|     by (rule Rats_abs_nat_div_natE)
|12|   hence "real (m^2) = ?x^2 * real (n^2)" by (auto simp add: power2_eq_square)
|13|   hence eq: "m^2 = 2 * n^2" using of_nat_eq_iff power2_eq_square by fastforce
|14|   hence "2 dvd m^2" by simp
|15|   hence "2 dvd m" by simp
|16|   have "2 dvd n" proof
|17|     from 2 dvd m obtain k where "m = 2 * k"
|18|     with eq have "2 * n^2 = 2^2 * k^2" by simp
|19|     hence "2 dvd n^2" by simp
|20|     thus "2 dvd n" by simp
|21|   qed
|22|   with 2 dvd m have "2 dvd gcd m n" by (rule gcd_greatest)
|23|   with lowest_terms have "2 dvd 1" by simp
|24|   thus False using odd_one by blast
|25| qed
|26| end
|27|
|28|
```

What is ITP'ing?

- Notable applications in formal mathematics
 - four-colour theorem [Gonthier 2007]
 - odd-order theorem [Gonthier et al. 2013]
 - Kepler's conjecture [Hales et al. 2015]
 - 98% of the “most interesting” 100 theorems
 - The Liquid-Tensor Experiment

What is ITP'ing?

- In CS formal proof is used for *formal verification*:
 - A system and its desired properties are specified in a formal language
- Notable application in CS:
 - To get reliable implementations
 - * SeL4: Linux Kernel [Klein et al. 2009]
 - * CompCert: compiler for C [Leroy 2009]
 - * CakeML: compiler for a flavour of ML [Kumar et al. 2014]
 - * model-checker [Esparza et al. 2013]
 - * SAT-solver [Blanchette et al. 2018]
- Adopted by industry:
 - Intel, Arm, AWS, Apple

What is ITP'ing?

- Some of my own **research**:
 - verification of AI and ML
 - formalisation of mathematics, esp. complexity theory and combinatorial optimisation
- If that interests any of you for a final year project, let's have a chat!

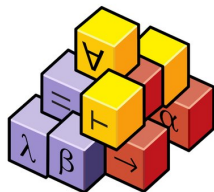
What will we do (i.e. learning outcomes)?

- In the next three lectures:
 - You will formally prove properties of simple functional programs
 - You will do this pen-and-paper and using the ITP Isabelle/HOL
- Take your own notes from the videos!

Expected Background Knowledge

- Mathematical induction (e.g. passed FC1 or FC2)
- First-order logic and how to formulate statements in it (e.g. ELA)
- Basic functional programming (e.g. passed PEP, esp. the Scala part)
- I will refresh them!

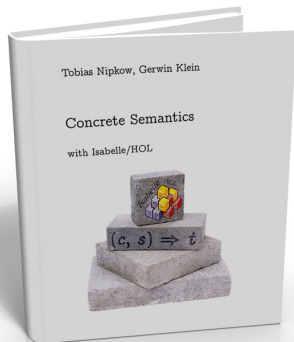
Organisational Points



- We will use the **Isabelle/HOL** theorem prover
 - Developed by Cambridge, TU Munich, UNSW, and many others
- Questions: KEATS+**Isabelle's Zulip channel**

Organisational Points

- First half of **Concrete Semantics**
- A brief Isabelle/HOL documentation: **Programming and Proving**



Organisational Points

- Assessment:
 - Coursework: an Isabelle assignment, deadline on 13th December
 - Exam questions: pen-and-paper proofs of properties of a functional program